

SAGE: 基于接地执行的智能家居代理

Dmitriy Rivkin, Francois Hogan, Amal Feriani, Abhisek Konar, Adam Sigal, 刘学, Gregory Dudek

摘要——大语言模型（LLMs）的常识推理能力和海量通用知识，使其成为在智能家居助手场景中解读用户请求的天然选择。然而，大语言模型缺乏关于用户及其住宅的具体知识，这限制了其潜在影响力。SAGE（基于接地执行的智能家居代理）通过采用一种方案克服了这些及其他限制，在该方案中，用户请求会触发一个由大语言模型控制的离散行动序列。这些行动可用于检索信息、与用户交互或操控设备状态。SAGE通过一个动态构建的大语言模型提示树来控制此过程，该提示树帮助其决定下一步采取哪个行动、判断行动是否成功以及何时终止流程。SAGE的行动集增强了大语言模型的功能，以支持智能家居助手的一些最关键需求。这些需求包括：灵活且可扩展的用户偏好管理（“我的球队今晚有比赛吗？”）、通过API读取访问任何智能设备的全部功能而无需设备特定代码（“调低我烘干机的屏幕亮度”）、持续的设备状态监控（“提醒我在打开冰箱时把牛奶扔掉”）、仅使用房间照片即可实现的自然设备指代（“打开梳妆台上的灯”）等等。我们引入了一个包含50项新颖且具有挑战性的智能家居任务的基准测试，其中SAGE取得了75%的成功率，显著优于现有的基于大语言模型的基线方法（30%的成功率）。

索引术语——自主大语言模型智能体、智能家居、物联网、生成式人工智能、具身人工智能、个性化人工智能、人工智能助手。

I. 引言

本文提出了一种方法，通过在顺序行动框架内对大语言模型进行增强，来释放大语言模型在智能家居助手的全部潜力。SAGE（基于接地执行的智能家居代理）通过动态生成的大语言模型提示树来指导这一顺序过程，这些提示整合了对正确解释和执行用户请求至关重要的相关信息。这被称为一种“自主大语言模型智能体”[1]策略，因为它允许大语言模型拥有比早期更僵化的框架（例如 [2] 和 [3]，）更高度的自主性，在那些早期框架中，大语言模型被用于在预定义的决策管道中回答一个或多个问题，但无法指导流程的控制流。可用行动集中的行动被称为“工具”。

工具既有输入（参数）也有输出。大语言模型被提示决定使用哪个工具（及其参数）。在内部，一个工具可以查询数据库、控制设备、与用户交互，或执行包括构建和执行其他大语言模型提示在内的多种行为。当工具执行完毕，其输出将被用于构建后续的提示。SAGE 通过构建工具选择提示来实现对大语言模型的增强，

工具接口、工具实现和工具输出格式。借助这些增强功能，SAGE 能够克服大语言模型固有的缺陷，包括但不限于缺乏对用户、其家庭及其设备的特定了解。整体方法在第二节中有更详细的描述。第三节介绍了最重要的工具，并描述了它们的功能和实现。

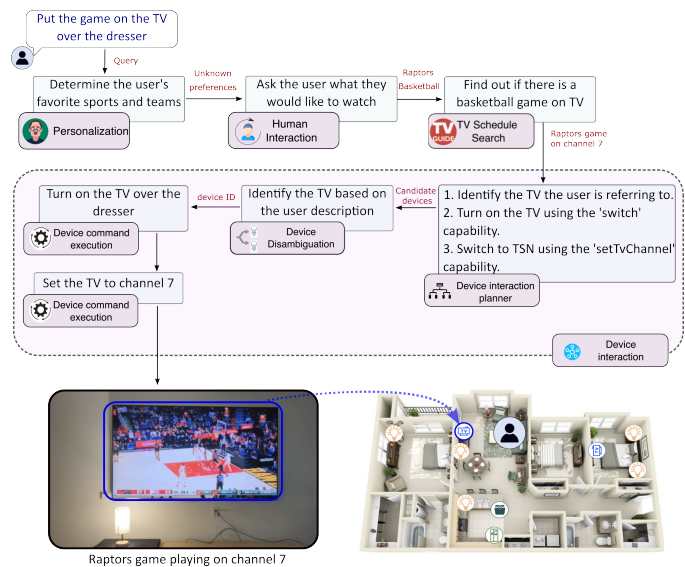


图1. 交互式演示，用户向SAGE询问：“打开梳妆台旁边的游戏”。该图说明了SAGE为完成任务所使用的工具序列。请注意，每个控制流决策都是由大语言模型做出的，而非手写逻辑代码。控制流在第四节中有逐步描述。此演示是使用真实的支持SmartThings的设备执行的。

SAGE设计的最终成果是一个主要由大语言模型控制、但不受其固有局限限制的决策流程，使用户与智能家居的交互达到了前所未有的灵活性和自然度。图1展示了这样一个决策过程的示例。乍看之下，这个示例可能显得平淡无奇，但关键在于，其中的每一步都是由大语言模型决定的，没有任何手动定义的决策逻辑。由于不存在手动决策逻辑，用户便不会局限于设计者预想的使用场景。因此，SAGE代表了相对于传统解决方案（如Bixby、Alexa、Google Assistant等）乃至此类产品的下一代大语言模型增强版本（例如[2]）的一次重大范式转变。

通过SAGE，我们的目标是超越当今智能家居中常见的简单、高度结构化、显式的语言要求，从而提升用户体验。

助手，因此我们设计了一个评估套件来测试其执行此任务的能力。该基准测试由50项具有挑战性的任务组成，并配有测试工具和验证逻辑，使得测试无需硬件（或人工评估员）介入即可运行。详细信息见第五节。我们在第六节中使用多种大语言模型评估SAGE，并将其与两个基于大语言模型的基线进行比较，结果显示性能提升超过2倍。

本文的主要贡献如下：

- 1) **个性化工具**：一个基于用户交互历史记录来回答关于用户偏好问题的模块。
- 2) **设备交互工具**：一个通过向大语言模型提供其API文档来与任何智能设备进行交互的模块。
- 3) **设备消歧工具**：一种机制，允许用户以自然、非刻板的方式（就像他们通常对另一个人说话那样）来指代他们的设备，其依据是家中该设备的单张照片。
- 4) **通过代码编写实现持久命令处理**：一种方案，允许大语言模型通过利用其代码编写能力来持续监控设备状态。
- 5) **SAGE**：将这些工具及其他工具集成到一个具有单一入口点的连贯系统中，从而克服大语言模型在智能家居环境中的关键固有局限。
- 6) **性能基准测试**：一个包含50项任务的基准测试，用于验证SAGE的性能。

二、系统概述

SAGE的顺序决策过程结构在算法1中描述。一些工具本身也是通过选择其他工具的顺序决策过程来实现的。如前所述，此类过程通常被称为“智能体”，例如在 [1]中。因此，我们将通过此类序列实现的工具称为“智能体工具”，并由一个顶层的智能体工具为用户交互提供入口点。

这些 d_i 功能是通过大语言模型实现的。用于决策函数 d_i 的大语言模型提示 $\mathcal{P}_i$ 构建如下：

$$\mathcal{P}_i = \text{Concatenate}(\text{TaskInfo}_i, \text{ToolInstructions}(S_i), \text{FormatInfo}, a, \text{HistoryInfo}(H_i))$$

其中：

- 任务信息 i 提供了关于要由元工具执行的任务的上下文信息。
- 工具指令(S_i)包含关于如何在元工具 m_i 可用的工具子集 S_i 中使用每个工具的说明。
- 格式信息指定了关于输出应如何格式化的说明，以便于解析。
- a 是元工具接收到的输入。
- 历史信息(H_i)将元工具的决策历史编译成大语言模型能够理解的格式。

算法1 SAGE 决策流程

要求：所有工具集合 $T = \{t_1, t_2, \dots, t_n\}$

要求：智能体工具子集 $M = \{m_0, m_1, \dots, m_p\} \subseteq T$
其中 m_0 为固定的入口点代理工具。

要求：决策函数集合 $D = \{d_0, d_1, \dots, d_p\}$,
其中 d_i 对应于 m_{i0}

要求：子集 $S_i \subseteq T$ ，即智能体工具可用的工具 m_i 。

要求：可能参数全集 A 。

要求：特殊的终止动作 τ ，表示一个智能体工具应终止其进程并返回其输出。

- 1: 为入口点智能体初始化决策历史 H_0
工具的决策函数 d_0 。
- 2: **循环** ▷ 等待用户输入的无限循环
- 3: 等待并接收用户输入 $userInput \in A$
- 4: $output \leftarrow \text{CALLAGENTTOOL}(m_0, H_0, \text{用户输入})$
- 5: 使用 $output$ 响应用户。
- 6: **结束循环**
- 7: **function** $\text{CALLAGENTTOOL}(m_i, H_i, a \in A)$
- 8: **loop**
- 9: $(action, output) \leftarrow d_i(H_i, input)$
- 10: **如果** $action = \tau$ **则**
- 11: $H_i \leftarrow \text{emptylist}$
- 12: **返回** $output$
- 13: **else**
- 14: 将动作解析为 (s, a^*) ，其中 $s \in S_i, a^* \in A$
- 15: **如果** $s \in M$ **则**
- 16: 识别索引 j ，使得 $s = m_j$
- 17: **如果未** H_j **初始化则**
- 18: 将 H_j 初始化为一个空列表
- 19: **结束如果**
- 20: $o \leftarrow \text{调用智能体工具}(m_j, H_j, a^*)$
- 21: **else**
- 22: $o \leftarrow s(a^*)$
- 23: **结束如果**
- 24: $H_i \leftarrow H_i + [(s, a, o)]$
- 25: **结束如果**
- 26: **结束循环**
- 27: **结束函数**

给定提示 $\mathcal{P}_i$ 后，随即对大语言模型进行采样：

$$(response, probability) \leftarrow \text{SampleLLM}(\mathcal{P}_i)$$

此处，大语言模型采样表示使用生成的提示词 $\mathcal{P}_i$ 查询语言模型的过程。随后，语言模型会返回一个响应以及一个相关的概率值，该值表示模型对其生成响应的置信度。大语言模型生成的输出需要被解析成适合算法的格式：

$$(action, output) \leftarrow \text{解析大语言模型输出}(response)$$

ParseLLMOutput 是一个函数，用于将大语言模型的响应解析为定义的 $action$ （要么是 τ ，要么是一个元组 (s, a) ，其中 s 是一个工具， a 是一个参数）以及关联的 $output$

表1 SAGE 工具层次结构。每一行对应 SAGE 中的一个工具，并指明其功能类别、名称、是否为智能体工具，以及可访问的子工具（如有）。

类别	Name	智能体工具	子工具
入口点	SAGE	yes	个性化、设备交互、条件代码编写、条件轮询、电子邮件和日历，天气，电视节目表搜索
个性化	个性化	no	N/A
	人机交互	no	N/A
	设备交互	yes	设备交互规划器，API文档检索，设备属性检索，设备命令执行，设备消歧
设备交互	设备交互规划器	no	N/A
	API文档检索	no	N/A
	设备属性检索	no	N/A
	设备命令执行	no	N/A
	设备消歧	no	N/A
监控	条件代码编写	yes	设备交互规划器，API文档检索，设备消歧，代码执行
	代码执行	no	N/A
	条件轮询	no	N/A
外部交互	电子邮件与日历	yes	获取联系人、创建日历事件、列出日历事件、创建电子邮件草稿、发送电子邮件消息，搜索电子邮件，获取电子邮件消息，获取电子邮件会话
	获取联系人	no	N/A
	创建日历事件	no	N/A
	列出日历事件	no	N/A
	创建电子邮件草稿	no	N/A
	发送电子邮件消息	no	N/A
	搜索电子邮件	no	N/A
	获取电子邮件消息	no	N/A
	获取电子邮件会话	no	N/A
	天气	no	N/A
	电视节目表搜索	no	N/A

这些数据在算法上下文中具有意义。如果大语言模型未能遵循格式信息中的指令，则无法完成解析，并会抛出错误。

结合这些步骤，使用大语言模型的决策函数 d_i 可以封装为：

$$d_i(H_i, input) = \text{解析大语言模型输出(大语言模型采样}(P_i))$$

在此框架内，规划（将高层目标分解为子步骤的过程）通过一种称为“思维链” [4]的技术实现。在这种方法中，工具指令和格式信息鼓励大语言模型在继续具体说明如何执行计划步骤之前，先输出一个计划。图2提供了一个高度简化的示例。请注意，决策函数 (d_i) 首次被调用时输出的多步骤计划，会通过历史记录 (H_i) 持久保存，以便智能体工具能够在多次大语言模型调用中继续执行同一计划。它也可能根据意外结果调整计划，从而促进故障恢复。

在本工作中，格式信息和解析大语言模型输出取自 ReAct [5]。历史信息函数是一个拼接操作。可能的参数集合 A 是所有字符串的集合。任务信息、工具指令、集合 T 中所有工具的实现，以及将子工具分配给元工具 S_i ，共同构成了被称为SAGE的方法。第三节描述了几个关键工具的实现。

III. 工具

在本节中，我们介绍为 SAGE 开发的一系列工具（完整列表参见表 I）。此外，

除了列出所有工具外，该表还标明了哪些是智能体工具，以及这些智能体工具的子工具，从而提供了整个工具层次结构的概览。为了帮助组织这些工具，我们将其分为 4 个功能类别：个性化（考虑用户偏好）、设备交互（与智能设备交互）、监控（持续监控设备状态以处理持久命令）以及外部交互（与智能家居外部的应用程序接口交互，例如电子邮件、天气）。

本节其余部分将详细介绍个性化、设备交互和监控类别中工具的实施方案。为简洁起见，外部交互工具的细节在此省略，因为此类工具在先前的工作中（例如在 Langchain [6] 中）已有充分探讨。

A. 个性化

本节介绍具有个性化相关功能的工具：个性化工具和人机交互工具。如图3所示，个性化工具由两个主要组件构成：(i) 存储所有过往用户交互历史记录的记忆，以及 (ii) 构建用户偏好分层理解的用户画像器。我们首先描述这两个组件，然后说明它们如何集成。最后，我们将简要描述人机交互工具。

1) 长期记忆：长期记忆 [1]，（如图3左下角所示）存储有关用户过去交互和行为的信息。该记忆记录用户指令命令和反馈的历史记录。类似于现有用于LLM增强的信息检索技术 [7]，

提示:**[任务信息]**

你是一个帮助用户开关灯的AI。在执行前，请逐步思考需要做什么。

[工具指令]**工具:**

- 灯具ID工具 (输入: 灯具常用名)
- 开灯工具 (输入: 灯具UUID)
- 关灯工具 (输入: 灯具UUID)

[格式信息]

请使用以下输出格式 思考: 处理请求需要执行的步骤 行动: 接下来要使用的工具 或 τ 行动输入: 工具的输入参数 或最终回复 (如果行动是 τ)

[a]用户输入: 打开装饰灯

[历史信息]

先前的大语言模型回复思考: 首先我需要使用灯具ID工具来找到装饰灯的UUID, 然后我需要使用开灯工具来打开它。行动: 灯具ID工具 行动输入: 装饰灯 先前的工具输出观察: "装饰灯"的UUID是 12e4df...bc4 回复: 思考: 我现在知道了装饰灯的UUID。现在我需要使用开灯工具来打开它。行动: 开灯工具 行动输入: 12e4df...bc4

图2. 简化的大语言模型提示构建与响应示例。蓝色文本表示提示，橙色文本表示响应，黑色文本是对下方提示部分的解释。在此示例中，决策函数(d_t)正在被第二次调用，因此历史记录(H_t)中填充了第一次调用的结果。

长期记忆用于检索与用户查询相关的记忆，以增强个性化工具可用的上下文信息。记忆中的每个条目都使用稠密检索嵌入模型 [8]进行编码。向量表示随后被索引并存储在向量数据库中。在本工作中，我们使用MiniLM嵌入模型 [9]。

2) 用户画像器: 用户画像，如图3右下角所示，提供了一个关于用户与智能体之间交互的高层次摘要，旨在动态、全面地理解用户偏好。我们采用了一种分层方法（最初在 [10], 中提出）来构建用户画像。用户画像器首先按日期对所有记忆条目进行分组，并生成每日用户偏好摘要，以高粒度捕捉用户偏好。接着，将每日摘要汇总成一个单一的全局摘要。

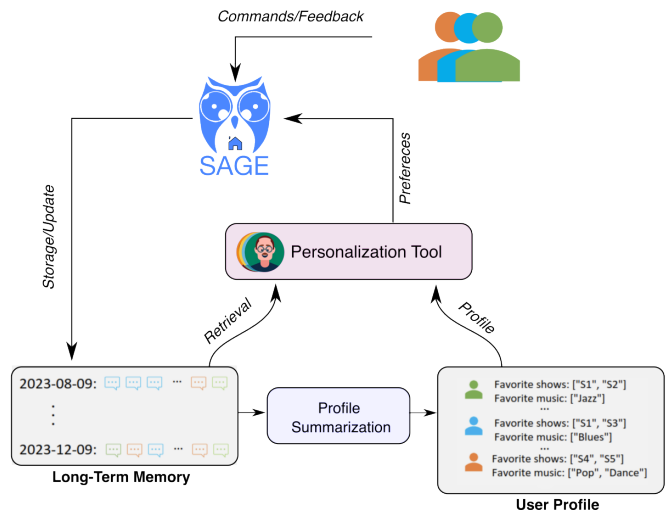


图3. 个性化工具概览。长期记忆存储并检索所有过去的用户话语。用户画像包含关于用户偏好的更普遍趋势。

该全局摘要即作为用户画像。我们选择分层方法主要基于两个原因: (i) 可扩展性: 随着长期记忆随时间增长，分层方法具有高度可扩展性，因为它适用于MapReduce式处理 [11]; (ii) 信息损失: 直接从长期记忆中生成简洁摘要需要很长的上下文提示。已知大语言模型在输入上下文中成功识别相关信息的能力会随着输入上下文长度的增加而下降 [12]。

3) 个性化工具: 个性化工具（图3）可以利用长期记忆和用户画像器来回答关于用户偏好的问题。当被问及关于用户偏好的问题时，该工具首先使用稠密检索嵌入模型将问题编码到与记忆相同的嵌入空间中。然后，在嵌入空间中使用余弦相似度作为距离度量，从长期记忆中检索出相似的记忆。接着，该工具构建一个大语言模型提示，其中包含检索到的记忆、用户画像以及问题。最后，它用该提示查询大语言模型并返回响应。用户画像和检索到的记忆是互补的；检索模块识别出一小部分提供狭窄而精确信息的候选记忆，而用户画像则呈现了用户更全面的视图。通过维护独立的画像和长期记忆来支持多用户。用户身份通过语音识别软件 [13]进行识别。

4) 人机交互工具: 人机交互工具允许 SAGE 向用户提问，通常用于澄清意图。该工具通过一个字符串调用，该字符串会通过文本转语音接口传达给用户。该工具会阻塞，直到用户回复，并使用现成模型（如 [14]）将用户的语音转录为文本。根据经验，我们发现引入用户交互工具可能导致智能体变得过于谨慎，过度使用此工具而非其他数据源来减少不确定性。我们在 SAGE 中使用提示工程来鼓励其不要过度使用该工具，

但如何最佳地权衡个性化、人机交互和风险规避，仍是一个活跃的研究课题。

B. 设备交互

在本节中，我们将介绍 SAGE 用于与智能设备交互的一系列工具。这些工具实现了灵活且可扩展的设备交互，其可扩展性体现在：新设备可以以可忽略不计的额外开发工作量集成到系统中，并且其功能可以得到最大程度的利用，而无需编写设备特定的代码。例如，如果用户将一台智能冰箱添加到其智能家居生态系统中，SAGE 可以将冰箱的存在及其所有功能（例如温度设置、开门检测、功耗等）整合到其决策过程中，而无需编写任何针对冰箱的特定代码。

大多数智能家居系统（如 SmartThings、Home Assistant、Google Home、Alexa 等）都提供了用于与用户家中的智能设备交互的应用程序接口。这些应用程序接口的文档可在网上找到，并且有使用它们的代码示例，这意味着在网页数据上训练的大语言模型（例如 GPT4）很可能对这些应用程序接口具有一些固有的了解。然而在实践中，我们发现大语言模型常常无法成功使用这些应用程序接口，原因是一些小错误，例如忘记需要检索的属性（attribute）的确切名称。此外，一些设备具有自定义功能，这些功能在网上没有可用的文档，只能通过查询设备的应用程序接口来获取。如果能在提示词中注入应用程序接口使用的详细信息，这些挑战是可以克服的，但为所有已连接的设备注入完整的文档是不可行的，因为这样做将远远超出当今大语言模型的最大提示词长度限制。

受LangChain的OpenAPI工具包启发，[15]设备交互工具被实现为一个智能体工具。该智能体工具仅利用设备及其相关功能的一般描述来生成高层计划，检索计划所需功能子集的详细文档，然后利用这些文档来构建API调用。此行为通过下文详述的一系列工具得以实现。

设备交互规划器工具：生成一系列必须由设备交互智能体工具执行的步骤，以完成给定的命令。该工具通过单次大语言模型查询实现。此查询包含一个设备列表、它们的功能以及每个功能作用的简短描述。它还包括输入指令以及关于计划应如何构建的描述。查询指定生成的计划的每个步骤应包含一个或多个设备ID、一个或多个功能，以及对该步骤需要做什么的自然语言描述。如果规划器无法根据其获得的信息直接推断出正确的设备，它可以向设备消歧工具（详见下文）提供多个候选设备和/或功能，以决定正确的选项。设备消歧工具会为每个提议的功能检索详细的文档，为智能体提供足够的信息以做出最终选择。使用规划工具，而非

依赖于设备交互智能体工具的思维链规划（如第二节所述），其动机在于此规划过程需要向提示词中注入大量信息。若将所有信息直接添加到设备交互智能体工具的提示词中，将导致大语言模型查询成本显著增加，因为智能体工具提示词在单次使用设备交互工具的过程中通常会被调用多次。

API文档检索工具：检索所请求设备功能的文档。该文档在可用时从网络抓取，否则通过API从设备获取。虽然从设备API提取的文档通常缺乏详细的使用自然语言描述，但它包含了属性、命令和命令参数的名称，其中许多含义仅从名称即可推断。此工具以功能列表作为输入，并以JSON格式返回每个功能的详细文档。使用JSON格式是为了方便，因为这是文档抓取器和设备API返回的格式。

设备属性检索工具和设备命令执行工具：这些工具允许智能体通过API进行通信，以读取属性和执行命令。我们将这些工具实现为围绕SmartThings REST API的封装器，用于查询和修改设备状态 [16]。为了使用这些工具，必须首先调用文档检索工具，以获取能力详情并正确格式化输入。请注意，如果输入格式不正确导致API抛出异常，我们根据经验发现，若将此异常相关的文本信息传回设备交互工具智能体，它通常能够做出反应并相应地修正API请求。

设备消歧工具：允许系统在存在多个同类设备实例（例如多个智能灯）的场景下，解析用户想要控制的具体设备。我们提出一种方法，通过利用视觉上下文来确定与任务相关的设备。通过在设备初始设置时拍摄其周围环境的照片，我们可以解析出设备ID，而无需用户硬编码唯一的设备名称（这很容易被遗忘，且访客可能不知道）。例如，在图4中，仅从图片就能明显看出灯位于餐厅。设备身份通过视觉语言模型进行消歧，如图4所示，其中使用了一个多模态VLM（OpenClip ViT-B-32 laion2b s34b b79k，[17]）来计算用户对设备的自然语言描述以及每张设备照片的嵌入向量。选择其图像嵌入向量与设备描述文本嵌入向量余弦相似度最高的设备。

C. 监控

本节介绍一组与设备状态监控功能相关的工具。许多更强大的智能家居行为，正是通过监控某些设备的状态并对状态变化做出反应的能力来实现的。这些行为被称为持久命令 [2] 或例程，

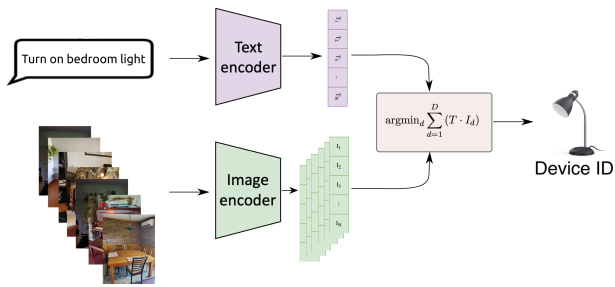


图4. 设备消歧。一种使用视觉语言模型来解析最符合用户自然语言设备描述的设备的方法。

因为系统应在条件事件发生后持续地以期望的方式运行（例如，每当早晨闹钟响起时，咖啡机就应启动）。智能家居解决方案通常通过IFTTT¹（IF This Then That）等应用程序中的条件语句来处理此问题。一旦条件被定义，就可以使用低成本的计算资源来执行条件检查。这种方法的一个缺点是系统缺乏灵活性，因为IFTTT例程依赖于必须由制造商预定义并由用户手动激活的条件触发器。

高度灵活的持久化命令处理可以在SAGE架构内实现，只需定期以持久命令作为输入运行SAGE即可。每次运行时，智能体可以检查命令是否已满足，如果满足则执行期望的行为。这种方法保留了智能体架构的所有功能且易于实现，但其缺点是需要智能体持续运行，从而产生显著的计算成本。

为了在最小化成本的同时增加系统的灵活性，我们提出了一种方法，使SAGE能够通过编写实现条件检查逻辑的Python代码来自编程条件例程。为此引入了两个工具：条件代码编写工具和条件轮询工具。SAGE智能体查询条件代码编写工具以编写必要的代码，然后将此代码注册到条件轮询工具，该工具会定期（每隔几秒）运行它。除了条件检查代码，它还会注册一个描述，说明当条件满足时必须采取的行动。一旦代码返回“True”，轮询过程就会触发SAGE智能体的第二次执行，并附带第一次执行时注册的命令。整个方法及其示例总结在图5中。

条件代码编写工具的实现，与设备交互工具面临着同样的挑战——即需要将API详情注入到生成代码的查询中。我们以类似的方式克服了这一挑战：创建一个智能体工具，该工具使用设备交互规划器工具和API文档检索工具。然而，与设备交互工具不同的是，它不使用设备属性检索和命令执行工具，

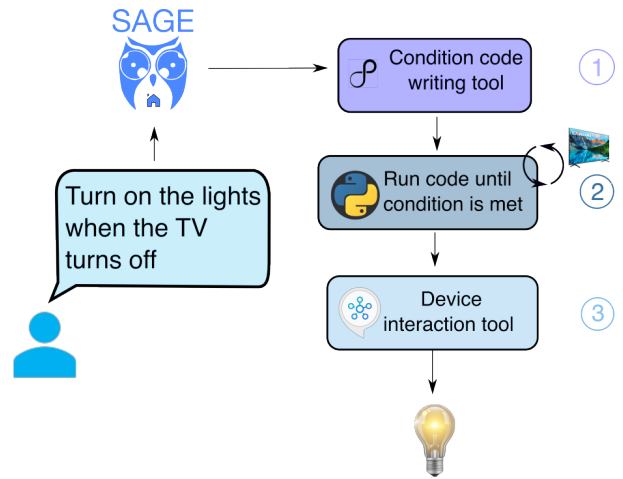


图5. 持久命令处理机制概述。1: 收到用户请求后，SAGE代理工具提取条件（“电视是否关闭？”）并使用条件代码编写工具编写代码来检查此条件。该工具将此代码注册到内存中，并返回函数名称（is_tv_off）。2: SAGE代理工具将函数 is_tv_off 与条件轮询工具一起注册，同时注册检测到条件后要执行的动作命令“开灯”。此时，SAGE代理工具执行完毕。条件轮询工具定期运行 is_tv_off 函数。一旦函数输出 True，条件轮询工具将使用已注册的动作“开灯”再次触发SAGE代理工具。请注意，仅当动作状态从 False 转变为 True 时才会触发智能体工具，以避免在电视关闭的整个期间重复运行智能体工具。3: 智能体工具开始执行命令“开灯”，并使用设备交互工具打开灯。

条件代码编写工具智能体工具则可以使用一个代码执行工具来测试其代码。此工具还会将运行过的代码存储在内存中，以便可以通过函数名进行引用。与设备属性检索和命令执行工具类似，代码执行工具通过将异常信息返回给代码编写智能体工具来处理异常，从而促进从错误代码中恢复。

四、执行示例

图1展示了SAGE代理工具处理单个命令的执行轨迹，说明了为完成任务而调用的一系列工具。在通过个性化工具检查长期用户交互记忆和用户档案信息后，系统未找到任何相关信息，因此必须调用人机交互工具直接询问用户的体育偏好。随后，SAGE代理工具使用电视节目表搜索工具找到相关的电视节目和频道。一旦用户的具体目标确立（将电视切换到特定频道），SAGE便调用设备交互工具，这是一个智能体工具。该设备交互工具首先调用设备交互规划器工具来确立必须执行的步骤序列。其生成的计划如下：首先，调用设备消歧工具来确定用户指的是哪台电视。其次，设备命令执行工具将电视打开。最后，再次使用设备命令执行工具来切换频道。设备交互工具成功执行此计划，然后将执行权交还给

¹<https://ifttt.com/>

顶层的SAGE代理工具，该工具响应用户并终止此次运行。

V. 评估

为了评估SAGE及竞争方法，我们创建了一个包含50个任务的数据集。测试任务的实现方式是：初始化设备状态和记忆，使用输入指令运行SAGE代理工具，然后评估设备状态是否被正确修改。对于涉及回答用户问题（而非修改设备状态）的任务，测试设计为：智能体必须检索一条特定信息（该信息不太可能被猜到）才能回答问题。随后使用一个基于LLM的评估器来检查答案是否包含预期信息。所有测试的结果均为二元（通过/失败）。

智能设备配置（设备类型、ID等）是通过配置一个包含真实SmartThings设备的家庭，然后保存这些设备的状态来创建的。这些设备包括：2台电视机、1台冰箱、1台洗碗机和4盏灯。设备的初始状态由每个任务的初始化例程进行修改。设备消歧工具中使用了真实设备在其实际位置的照片。

我们根据现有系统难以处理的五类挑战对测试用例进行分类。集合中的大多数测试都属于这些类别中的一个或多个。这些类别是：

- 1) **个性化**：整合用户偏好知识以正确解释请求。
- 2) **意图解析**：理解模糊命令并得出逻辑结论。
- 3) **设备解析**：根据自然语言描述识别目标设备ID。
- 4) **持久性**：处理需要持续监控系统状态的命令。
- 5) **指令链**：解析由多个指令组成的复杂命令，将其分解为可执行的步骤，并以连贯的方式执行每个步骤。

我们还加入了第六个类别，**直接指令**，用于表示那些执行起来更简单的测试用例，因为它们不包含上述5类挑战中的任何一项。

测试用例设计为完全自动化运行。因此，我们在测试期间禁用了人机交互工具。我们使用5种不同的大语言模型测试SAGE：GPT4、GPT4-turbo、GPT3.5-turbo (ChatGPT)²，Lemur [18]和Claude2.1³。对于所有大语言模型，我们将温度参数设置为0。对于每个大语言模型，我们每个测试用例都运行3次，因为即使温度为0，大语言模型的性能仍具有一定随机性。

除了主要的50个任务集之外，在SAGE开发完成后，我们还创建了10个额外的“测试集”任务。这些任务旨在验证提示词是否未针对该任务集进行过度工程化。开发这些任务的作者熟悉SAGE架构，但并未参与最终的提示工程阶段。

为了更深入地了解每个大语言模型面临的关键挑战，我们手动标注了在所有三次运行中持续出现的失败案例。这些失败根据表II中定义的术语进行分类。由于给定测试用例的成功取决于多个决策步骤，失败可能发生在大语言模型推理链的多个步骤中。在我们的分析中，给定测试用例的失败分类是基于执行过程中第一个错误。

我们通过将故障类别分组为层级来对其进行情境化。这些层级的组织方式是，大多数情况下，第 n 层级的故障意味着第1层到第 $n - 1$ 层是成功的。例如，如果标注者将一个测试用例标记为因规划失败（第2层故障）而失败，这意味着它在指令理解和格式方面是成功的（第1层故障）。

A. 基线方法

为了将SAGE的性能置于情境中，我们在测试任务上将我们的方法与两种基于大语言模型智能家居自动化基线方法进行比较。第一种方法称为“One Prompt”，它涉及创建一个包含用户指令以及所有设备状态的单一提示词，并要求大语言模型生成更新后的状态作为响应。完整的设备状态序列化为JSON格式后，会超过GPT4的令牌限制（8000个令牌），因此我们手动选择了测试中涉及的设备状态部分。此外，模型被要求输出需要进行的更改，而不是完整的新状态。

第二个基线名为“Sasha”，它实现了 [2] 中描述的流程，并做了一些修改。[2] 中的原始流程包含5个状态——澄清、过滤、规划、反馈和执行。澄清和反馈阶段需要人工干预，因此与我们完全自动化的测试框架不兼容，所以在我们的实现中移除了它们。此外，该流程区分了“传感器”和可操作设备，允许流程输出基于传感器的触发器-动作对来处理持久命令。这需要手动定义触发器，而我们的测试框架不支持此操作，因为SAGE能够通过编写代码来生成自己的触发器。因此，我们实现的Sasha不包含触发器概念，也因此无法处理持久命令。

这两个基线都处于劣势，因为它们无法整合SAGE所使用的不同信息来源（例如，用户偏好、设备照片等）。尽管如此，这些基线能让读者评估任务集的难度，并理解整合来自多种来源的信息能在多大程度上提升智能家居自动化系统的性能。我们没有提供一个能访问与SAGE相同信息的基线，因为据我们所知，目前没有先前的工作能够整合所有这些信息来源。

六、结果

三种方法——SAGE、Sasha 和 One Prompt——在 50 项任务集上的总体成功率如图

²<https://platform.openai.com/docs/models>
³<https://www.anthropic.com/index/claude-2-1>

表II 故障类别描述。故障等级列有助于理解故障类型类别。在大多数情况下，第N级的故障意味着较低等级的故障已被避免（4 = 最高，1 = 最低）。

故障等级	故障类型	描述
1	格式 指令理解	智能体未能遵循工具指令所要求的模板。 智能体未能理解用户的请求。
2	规划	智能体的计划不正确或缺少步骤。
3	计划执行 工具选择 工具种群 API请求 代码编写	智能体提出了正确的计划，但执行过程缺少步骤。 智能体在执行过程的某个阶段选择了错误的工具。 智能体为工具提供了错误的参数。 智能体发出了错误的SmartThings API请求（错误的属性、命令或组件）。 智能体生成的代码无法正常工作。
4	故障工具 大语言模型限制 幻觉	非智能体故障（通常是设备消耗工具中的视觉语言模型故障）。 大语言模型缺乏关键常识或上下文长度不足。 智能体虚构了不存在的概念（例如设备、组件、用户需求）。
NA	其他	所有其他失败情况。

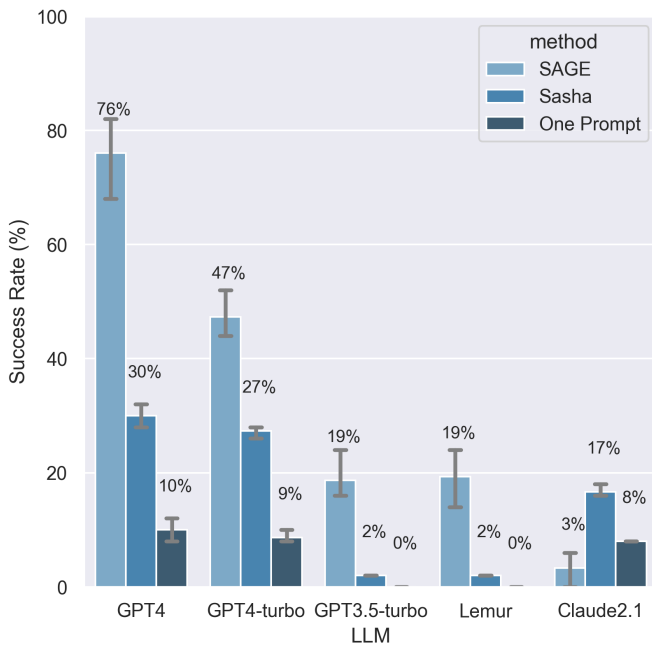


图 6. 在 50 项挑战性任务上的总体成功率。SAGE 利用一系列工具，能够将大量信息整合到其决策过程中，从而以显著优势超越其他基于大语言模型的方法。它主要为配合 GPT4 使用而设计，但在包括 Lemur 等开源模型在内的多种大语言模型上，其表现也优于其他方法。误差条表示三次运行中的最高分和最低分。

6 所示。SAGE 在使用 GPT4 时实现了 75% 的总体成功率，远超任一基线，这表明它确实能够通过使用其工具整合多种信息来源。不出所料，无论采用何种方法，GPT4 在所有大语言模型中均取得了最高性能。这很大程度上是因为它是最强大的模型，但也因为提示词是针对它优化的。GPT4-turbo 在 50 项任务集上的表现似乎明显逊于 GPT4，尽管据称其性能相近。对于这一现象，有几种可能的解释。首先，该模型的具体细节尚未公开，但基于其运行成本低数倍的事实，我们可以推测它比 GPT4 拥有更少的参数。其次，我们的经验表明，GPT4-turbo 更加

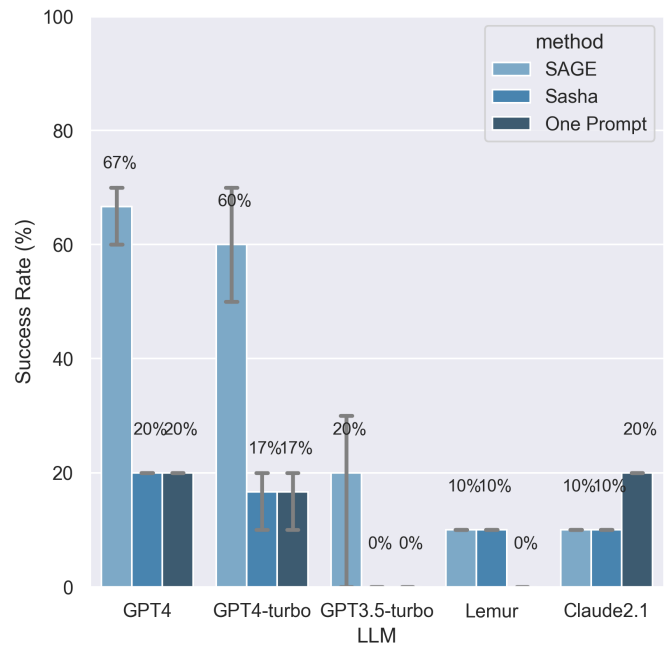


图 7. 在包含 10 项额外任务的“测试集”上的总体成功率。这些任务在迭代过程中未被 SAGE 的设计者见过，用于验证 SAGE 中使用的提示词是否未针对特定任务集进行过度优化。误差条表示三次运行中的最高分和最低分。

其对话性不如 GPT4，导致遵循指令的可能性较低，因此在智能体式任务上表现更差。最后，SAGE 提示词是针对 GPT4 优化的。如果我们投入同等精力针对 GPT4-turbo 进行优化，其性能很可能会显著提升。在图 7 所示的 10 项“测试集”任务中，两种模型取得了大致相当的性能。然而，鉴于该集规模较小，其在分辨性能差异方面可能不如主测试集准确。

从图 6 中观察到的另一个有趣现象是，Claude2.1 在 SAGE 中的表现非常差，甚至比两个基线模型都要差。这主要归因于 Claude2.1 无法遵循格式指令以及其糟糕的工具选择能力，正如表 III 所示，该表展示了手动故障分析的结果。要成功作为智能体的核心，一个大语言模型必须

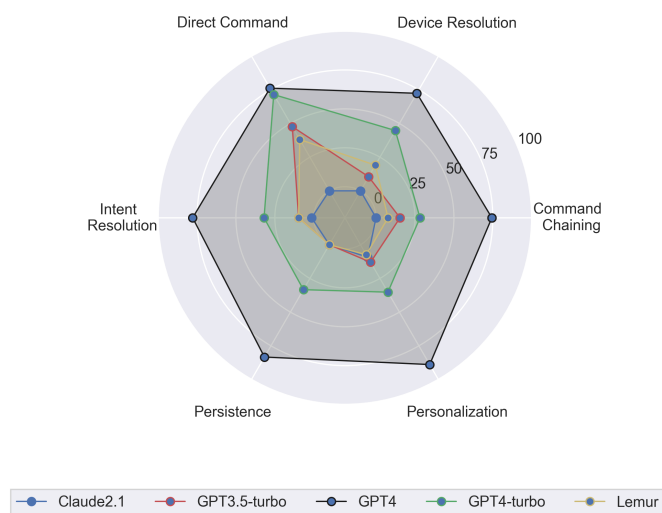


图8. SAGE在50个任务集上各类挑战性任务的成功率。请注意图表的内环表示零成功率，而非中心点。

能够遵循与格式和工具选择相关的指令，因为智能体必须能够将自然语言推理转化为具体的现实世界行动。

Claude2.1似乎没有显著的推理问题（从其在规划阶段没有失败可以看出），但其缺乏执行这一基本功能的能力，使其成为SAGE和其他智能体架构的糟糕候选者。

GPT3.5-turbo与Lemur表现相当，这对于那些有兴趣在智能体应用中使用开源大语言模型的人来说是一个令人鼓舞的结果。

对图8（展示了SAGE在各类挑战性任务上的成功率）的检查表明，采用GPT4-turbo的SAGE在不同挑战类别中保持了相当一致的性能。另一方面，能力较弱的大语言模型在直接指令上的表现往往远优于更具挑战性的任务。这证实了我们所识别的类别确实给智能家居自动化系统带来了显著困难。

A. 故障分析

表III总结了手动故障分析的结果。在本节中，我们将逐一回顾各个大语言模型，并就它们失败的主要原因提供一些讨论。

在分析的GPT4的9次失败中，有6次是由于误解了用户指令。这些指令有些模糊，大语言模型未能应用测试设计所预期的常识。另外3次失败是由于大语言模型使用了错误的设备组件（在SmartThings API中，某些设备有多个“组件”，例如智能冰箱的冷冻室部分和冷藏室部分各有一个组件）。

GPT4-turbo 的性能受到工具使用问题的阻碍，这些问题占失败案例的 69%（API 请求格式和工具种群）。确实，GPT4-turbo 最常因无法正确构建符合 SmartThings API 规范的请求而失败。这包括提供正确的参数类型、组件和功能。

GPT3.5-turbo 主要失败于无法生成合适的计划，并且在向工具传递正确的参数方面存在困难。它也通常无法利用可用设备状态来辅助其决策过程。此外，它常常无法正确解读用户的请求，而这通常需要较强的常识推理能力。

Lemur 难以根据当前设备状态的上下文正确解读用户指令，因此也难以生成有效的计划。Lemur 生成的计划缺失了关键步骤，例如使用设备消歧来找到正确的工具，或认识到需要为持久命令创建触发器。它还通过多轮试错来寻找正确的设备控制 API调用。

Claude2.1 常常难以遵循预期的响应格式。这涉及到先输出思考，再基于先前的观察输出行动。尽管我们向智能体提供了详细的错误日志和调试提示，Claude2.1 仍无法从其错误中恢复。Claude 另一个常见的故障类型是工具选择。例如，Claude2.1 经常在不需要的情况下，对直接指令使用个性化工具，而不是设备交互工具。请注意，工具使用是 Claude 最近新增且仍在开发中的功能。

七、结论

本文介绍了 SAGE，一种面向智能家居应用的大语言模型智能体架构。SAGE 在一个顺序决策过程中协调工具的使用。它集成了一系列新颖的工具，旨在应对智能家居自动化中的关键挑战，包括个性化、灵活的设备交互、设备监控和设备消歧。我们还创建了一个具有挑战性的智能家居自动化测试用例数据集，用于测试系统的个性化能力、从非结构化查询中解析用户意图的能力、解析以自然方式指代的设备（例如“钢琴旁的灯”）的能力，以及妥善处理命令持久性和链式操作的能力。这些挑战对当今的智能家居自动化系统而言非常困难，但也反映了智能家居用户对下一代系统所期望的复杂程度。

SAGE在这些任务上取得了约75%的成功率。这个数值虽然不完美，但远高于现有的家庭自动化解决方案，即使是那些基于强大LLM技术的方案。每一次成功都需要连续成功地使用多个工具，这意味着实际上成功使用工具的次数远多于失败的次数。为了理解失败的潜在原因，我们手动分析并归类了每一次失败。分析表明，如此复杂的决策过程可能出错的方式有很多，并且需要当今最强大的LLM才能达到可接受的性能。然而，专为智能体任务微调的开源模型Lemur所取得的可观性能令人鼓舞。我们预计，在一到两年内，SAGE搭配开源LLM的性能将与当今的GPT4相当，而成本仅为后者的一小部分。

表III 手动故障分析结果。此分析将故障归类为12种故障类型之一。故障等级列有助于理解故障类型类别的背景。在大多数情况下，第N级故障意味着避免了较低等级的故障（4 = 最高，1 = 最低）。仅分析了在所有三次运行中均一致失败的测试用例。

故障等级	故障类型	GPT-4	GPT-4 Turbo	GPT-3.5 Turbo	Lemur	Claude 2.1
		总故障数 = # 每次运行故障数 × 3				
		9	51	96	111	141
		故障率 (%)				
1	格式指令理解	0 67	0 14	12 2	0 0	35 1
2	规划	0	6	32	43	0
3	计划执行	0	2	5	10	2
	工具选择	0	4	8	3	37
	工具种群	0	18	27	24	15
	API请求格式化	33	51	0	0	1
4	代码编写	0	6	6	0	0
	故障工具	0	0	5	12	0
	大语言模型限制	0	0	1	7	0
NA	幻觉	0	0	0	0	2
	其他	0	0	0	1	7

因此，SAGE代表着迈向创建真正灵活的智能家居自动化系统的有希望的第一步，用户可以与这些系统进行交互，其自然程度如同与密友交流一般。

八、相关工作

A. 家庭自动化

一个智能家居系统由相互连接的物联网智能设备组成，能够实现对家居环境的同步监控、感知和控制。自动化控制这些设备可以提升生活质量、舒适度和资源利用效率 [19]。近期的研究致力于利用机器学习来增强智能家居系统的功能。例如，[20] 提出基于活动识别执行家庭自动化，开发了一种深度学习算法，用于从加速度计数据中识别用户活动。另一个重点是语音家庭助手，其任务是理解用户的语音指令。例如，[21]开发了一个基于自然语言处理和物联网的语音控制家居自动化系统，用于控制四种基本电器。借助先进的自然语言处理技术，当前诸如Bixby、Google Assistant和Alexa等商业解决方案提供了用户友好的界面，能够处理从购物、设置提醒到设备控制和家庭自动化等各种命令和问题。然而，这些现代家庭助手通常难以处理隐晦和复杂的命令 [22]。换言之，当用户的话语无法轻易映射到预设程序化例程时，这些系统往往会失败。

为了尝试克服其中一些挑战，最近的研究提出利用大语言模型的推理功能来更好地理解 and 执行用户指令。具体而言，Sasha [2]引入了在智能家居环境中使用大语言模型的方法，并展示了大语言模型可用于响应复杂或模糊的命令，产生合理的行为。Sasha实现了一个决策流程，其中每个步骤（例如选择要使用的设备，或检查某个例程是否存在）都是通过

一个大语言模型来实现的。然而，与SAGE不同，Sasha流程的各个阶段是手动定义且固定的，这限制了其灵活性。相比之下，SAGE依赖一个大语言模型来决定要执行的操作步骤序列。

如今的智能家居用户也可以选择手动定义IFTTT式例程，将触发器条件与操作连接起来 [23]，以创建复杂的行为。这种方法很不方便，因为它必须由用户通过应用程序或网页界面手动实现。它还存在一个限制，即触发器条件必须由设备制造商定义。用户也可以编写自己的应用程序来管理设备状态，例如使用SmartThings SmartApps[24]，但这需要超出大多数用户能力的技术水平，以及大量的时间投入。基于网络的服务，如IFTTT、Zapier⁴和Home Assistant⁵，使用户能够创建规则来控制他们的智能设备。这些服务的优势在于，它们简化了连接各种服务和智能设备的过程，无需广泛的编程知识。然而，这些解决方案也缺乏大语言模型所提供的推理和情境感知能力。

近期，IFTTT已开始通过创建ChatGPT插件⁶来利用大语言模型，该插件提供了一个更用户友好、更易访问的界面来与自动化平台交互。然而，该插件不允许ChatGPT生成新的例程，而只能触发现有例程。Zapier也提供了一项基于大语言模型的服务，允许用户以更自然的方式创建自动化流程，但目前其推理的复杂性尚有不足。[2] 创建了一个基于大语言模型的流程，可输出触发器-动作对以创建简单的IFTTT例程，但这些例程的逻辑复杂性以及触发器的灵活性均有限。早期的学术研究探索了训练序列到序列模型，以根据自然语言描述 [25]合成IFTTT或Zapier例程。结果令人鼓舞，但局限性在于序列模型能够生成待调用的

⁴<https://zapier.com/> ⁵

<https://www.home-assistant.io/> ⁶

<https://ifttt.com/explore/business/ifttt-ai>

函数序列，却无法生成这些函数所需的参数。

B. 自主智能体

LLM驱动的自主智能体旨在执行复杂多样的任务。这通常涉及将任务分解为多个阶段或子任务。文献中已提出了几种智能体架构设计[1]。思维链 [4]是一种著名的提示技术，它使智能体能够通过逐步规划和行动来执行复杂的推理。在思维链的实现中，提示中会插入几个思维链演示来指导智能体的推理过程。或者，零样本思维链 [26] 通过在提示中简单添加“逐步思考”这句话，展示了大语言模型的推理功能。另一项工作通过采用树状推理结构扩展了思维链，其中每个中间步骤都可以有自己的一组子步骤（例如 [27]）。上述工作在计划生成过程中没有考虑环境反馈（即已采取行动的结果）。后续工作确实纳入了这种反馈。例如，ReAct智能体 [5]结合了采取行动后从环境（例如，API调用或工具的结果）收到的观察结果。这些观察结果在后续每个推理步骤中都会被考虑进去。人类反馈也可以通过询问更多细节、偏好等来帮助智能体调整和完善其计划。

智能体设计的另一个重要部分是使用外部工具来执行动作。这使得智能体能够超越其内部知识。应用程序接口是最常见的工具类型，而大语言模型（例如 Gorilla [28] 和 ToolLLM[29]）已专门针对应用程序接口的使用进行训练。除了应用程序接口，外部知识库（例如文档数据库）也可作为一种工具，用于获取特定信息或专家知识 [30]。

参考文献

- [1] L. Wang, C. Ma, X. Feng, Z. Zhang, H. ran Yang, J. Zhang, Z.-Y. Chen, J. Tang, X. Chen, Y. Lin, W. X. Zhao, Z. Wei, and J. rong Wen, “基于大语言模型的自主智能体综述,” *ArXiv*, vol. abs/2308.11432, 2023. [在线]. 可访问: <https://api.semanticscholar.org/urn:681004713>.
- [2] E. King, H. Yu, S. Lee, and C. Julien, “Sasha: 智能家居中基于大语言模型的创造性目标导向推理,” *arXiv preprint arXiv:2305.09802*, 2023. [3] Zapier, “Zapier,” <https://actions.zapier.com/>. [4] J. Wei, X. Wang, D. Schuurmans, M. Bosma, F. Xia, E. Chi, Q. V. Le, D. Zhou 等, “思维链提示激发大语言模型中的推理,” *Advances in Neural Information Processing Systems*, vol. 35, pp. 24 824–24 837, 2022. [5] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafraan, K. Narasimhan, and Y. Cao, “ReAct: 在语言模型中协同推理与行动,” *arXiv preprint arXiv:2210.03629*, 2022. [6] H. Chase, “Langchain,” <https://github.com/langchain-ai/langchain>. [7] Y. Zhu, H. Yuan, S. Wang, J. Liu, W. Liu, C. Deng, Z. Dou, and J.-R. Wen, “用于信息检索的大语言模型: 一项综述,” *arXiv preprint arXiv:2308.07107*, 2023. [8] V. Karpukhin, B. O’guz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W.-t. Yih, “开放域问答的密集段落检索,” *arXiv preprint arXiv:2004.04906*, 2020. [9] W. Wang, F. Wei, L. Dong, H. Bao, N. Yang, and M. Zhou, “Minilm: 用于预训练Transformer任务无关压缩的深度自注意力蒸馏,” *CoRR*, vol. abs/2002.10957, 2020.
- [10] W. Zhong, L. Guo, Q. Gao, and Y. Wang, “Memorybank: 增强大语言模型的长期记忆,” *arXiv preprint arXiv:2305.10250*, 2023. [11] J. Dean and S. Ghemawat, “MapReduce: 简化大型集群上的数据处理,” *Commun. ACM*, vol. 51, no. 1, p. 107–113, 一月 2008. [12] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, “迷失在中间: 语言模型如何使用长上下文,” *arXiv preprint arXiv:2307.03172*, 2023. [13] “Vosk语音识别工具箱,” <https://github.com/alphacep/vosk-api>, 访问于: 2023-10-27. [14] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, “通过大规模弱监督实现鲁棒语音识别,” 2022 [] 2212 04356. 在线. 可访问: <https://arxiv.org/abs/>. “Langchain OpenAPI工具包,” <https://python.langchain.com/docs/integrations/toolkits/openapi>, 访问于: 2023-10-26. [16] “SmartThings API,” <https://developer.smarththings.com/docs/api/public>, 访问于: 2023-10-26. [17] G. Ilharco, M. Wortsman, R. Wightman, C. Gordon, N. Carlini, R. Taori, A. Dave, V. Shankar, H. Namkoong, J. Miller, H. Hajishirzi, A. Farhadi, and L. Schmidt, “OpenCLIP,” 2021年7月, 如果您使用此软件, 请按如下方式引用. [] 在线. 可访问: <https://doi.org/10.5281/5143773>. [18] zenodo. Y. Xu, H. Su, C. Xing, B. Mi, Q. Liu, W. Shi, B. Hui, F. Zhou, Y. Liu, T. Xie, Z. Cheng, S. Zhao, L. Kong, B. Wang, C. Xiong, and T. Yu, “Lemur: 协调自然语言与代码以构建语言智能体,” 2023. [19] C.-W. C. Ki, E. Cho, and J.-E. Lee, “智能个人助理 (IPA) 能成为你的朋友吗? IPA 与其用户之间的准友谊发展机制,” *Computers in Human Behavior*, vol. 111, p. 106412, 2020. [20] R. D. Manu, S. Kumar, S. Snehashish, and K. Rekha, “使用物联网和深度学习的智能家居自动化,” *International Research Journal of Engineering and Technology*, vol. 6, no. 4, pp. 1–4, 2019. [21] P. J. Rani, J. Bakthakumar, B. P. Kumaar, U. P. Kumaar, and S. Kumar, “使用自然语言处理 (NLP) 和物联网 (IoT) 的语音控制家居自动化系统,” 发表于 *2017 Third International Conference on Science Technology Engineering & Management (ICONSTEM)*, 2017, pp. 368–373. [22] E. Luger and A. Sellen, “就像有个很差的私人助理”: 对话智能体的用户期望与体验之间的鸿沟,” 发表于 *Proceedings of CHI 2016*, 2016. [23] H. Yu, J. Hua, and C. Julien, “数据集: 分析IFTTT配方以研究人类如何使用物联网 (IoT) 设备,” *CoRR*, vol. abs/2110.00068, 2021. [24] “SmartThings智能应用文档,” <https://developer.smarththings.com/docs/connected-services/create-a-smartapp>, 访问于: 2023-10-27. [25] D. Dalal and B. V. Galbraith, “评估用于If-Then程序合成的序列到序列学习模型,” *CoRR*, vol. abs/2002 03485. 在线. 可访问: <https://arxiv.org/abs/>. 2002 03485. [26] T. Kojima, S. S. Gu, M. Reid, Y. Matsuo, and Y. Iwasawa, “大语言模型是零样本推理器,” *Advances in neural information processing systems*, vol. 35, pp. 22 199–22 213, 2022. [27] S. Yao, D. Yu, J. Zhao, I. Shafraan, T. L. Griffiths, Y. Cao, and K. Narasimhan, “思维树: 大语言模型的深思熟虑问题解决,” *arXiv preprint arXiv:2305.10601*, 2023. [28] S. G. Patil, T. Zhang, X. Wang, and J. E. Gonzalez, “Gorilla: 连接海量API的大语言模型,” *arXiv preprint arXiv:2305.15334*, 2023. [29] Y. Qin, S. Liang, Y. Ye, K. Zhu, L. Yan, Y. Lu, Y. Lin, X. Cong, X. Tang, B. Qian, S. Zhao, R. Tian, R. Xie, J. Zhou, M. Gerstein, D. Li, Z. Liu, and M. Sun, “ToolLLM: 促进大语言模型掌握16000+ 真实世界API,” 2023. [30] Y. Ge, W. Hua, J. Ji, J. Tan, S. Xu, and Y. Zhang, “OpenAGI: 当大语言模型遇见领域专家,” *arXiv preprint arXiv:2304.04370*, 2023.